DocuScan

A Browser-Based Intelligent Document Scanner with AI-Powered Data Extraction

---

Final Year Project Report

[Department of Computer Science & Engineering]
[Your University Name]

Submitted To

[Supervisor Name]
[Designation, Department, University]

Submitted By

[Your Full Name]
Student ID: [Your ID] | Course: [Course Code]

[Month Year]

Declaration

I hereby declare that this project report titled "DocuScan: A Browser-Based Intelligent Document Scanner with AI-Powered Data Extraction" is the result of my own original work carried out under the supervision of [Supervisor Name]. This work has not been submitted previously for any degree or examination at this or any other institution.

All external sources, ideas, and materials that have been referenced or consulted in the preparation of this report have been properly acknowledged and cited in accordance with the academic conventions of [University Name].

I understand and accept the university's policies on academic integrity and plagiarism, and I confirm that this report complies with those policies to the best of my knowledge.

Student Signature: _____

Name: [Your Full Name]

Student ID: [Your ID]

Date: _____

Supervisor Signature: _____

Name: [Supervisor Name]

Designation: [Designation]

Date: _____

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my supervisor, [Supervisor Name], for their invaluable guidance, patience, and continuous support throughout this project. Their technical insights on image processing and AI integration, along with timely feedback during each stage of development, were instrumental in shaping this project into a complete and meaningful piece of work.

I extend my sincere thanks to the Department of Computer Science and Engineering at Hajee Mohammad Danesh Science and Technology University for providing access to the necessary academic resources, software tools, and computing infrastructure required for this project. The learning environment fostered by the department has been essential to my academic and technical growth.

I am also grateful to my colleagues and fellow students who participated in user testing sessions and provided candid feedback on the DocuScan application. Their observations helped identify usability gaps that I would not have noticed independently.

Special acknowledgement is due to the open-source community behind the key libraries and tools used in this project: the contributors to Tesseract.js, the React and Vite ecosystems, and the Dexie.js maintainers. Gratitude is also extended to the browser standards community whose work on the Canvas 2D API made lightweight, dependency-free image enhancement possible. These freely available, well-documented technologies made it possible to build a sophisticated application entirely in the browser.

Finally, I would like to thank Google for providing the Gemini 1.5 Flash API with a free access tier, which made AI-powered document understanding accessible for this academic project without requiring a financial investment.

Abstract

Document digitization has become an essential process in modern information management, yet the gap between physical documents and structured digital data remains a significant challenge for individuals and small organizations. Physical documents such as receipts, invoices, utility bills, and handwritten notes are perishable, difficult to organize, and require time-consuming manual data entry to convert into usable digital formats.

This report presents DocuScan, a browser-based intelligent document scanner that addresses the complete document digitization pipeline within a single, client-side web application. DocuScan eliminates the need for backend infrastructure by running all image processing operations locally in the browser using the native Canvas 2D API, while leveraging the Google Gemini API for AI-powered document understanding. The system captures document images via the device camera or file upload, applies an image enhancement pipeline using the Canvas 2D API to correct perspective distortion and enhance image quality, extracts text using Tesseract.js for in-browser OCR, and employs the Gemini 1.5 Flash multimodal model to classify the document type and extract structured data fields into a normalized JSON format.

The key technologies employed include React 18 with TypeScript for the user interface, the Canvas 2D API for lightweight, browser-native image preprocessing, Tesseract.js for optical character recognition with word-level bounding box output, the Google Gemini API for document classification and field extraction, and Dexie.js for local IndexedDB-based document archiving. Key features include image downscaling, auto-levels enhancement, custom sharpening, grayscale conversion, handwriting recognition via Gemini Vision, a searchable local document archive, smart auto-tagging, batch processing, and export in CSV, JSON, and PDF formats.

Testing across five document categories — printed receipts, invoices, ID cards, forms, and handwritten notes — demonstrated successful end-to-end processing. The system achieved accurate field extraction on printed documents under normal lighting conditions, with AI-powered structuring showing consistent extraction of key fields including vendor, date, total amounts, and line items. The project demonstrates that a powerful, privacy-respecting document intelligence tool can be built without cloud infrastructure, contributing to the growing body of work on accessible and affordable browser-based AI applications.

Keywords: *Document Scanning, OCR, Image Processing, Gemini API, Canvas 2D API, Tesseract.js, React, IndexedDB, Browser-Based Application*

Table of Contents

Declaration 2
Acknowledgements 3
Abstract 4
List of Figures 7
List of Tables 7
List of Abbreviations 8
Chapter 1: Introduction 9
1.1 Background and Motivation 9
1.2 Problem Statement 9
1.3 Objectives 10
1.4 Scope and Limitations 10
1.5 Report Organization 11
Chapter 2: Literature Review and Related Work 12
2.1 Overview of Document Digitization 12
2.2 Image Preprocessing Techniques for Document Scanning 12
2.3 Optical Character Recognition Systems 13
2.4 AI-Powered Document Understanding 13
2.5 Existing Document Scanner Applications 14
2.6 Summary 14
Chapter 3: System Design 16
3.1 System Architecture Overview 16
3.2 Technology Stack 17
3.3 Data Flow Design 18
3.4 Database Design 20
3.5 User Interface Design 20
3.6 AI Integration Design 22
Chapter 4: Implementation 23
4.1 Project Setup and Configuration 23
4.2 Image Capture Module 23
4.3 Image Preprocessing Pipeline 23
4.4 OCR Integration 25
4.5 AI Integration with Google Gemini API 25
4.6 Local Storage and Archive 26
4.7 Export Features 27
4.8 Key Implementation Challenges and Solutions 27
4.8.1 Canvas 2D Pipeline — Transition from OpenCV.js 27
4.8.2 Bounding Box Coordinate Scaling 27
4.8.3 Gemini Non-JSON Responses 27
4.8.4 Eliminated Load Latency via Canvas 2D Migration 28
4.8.5 Web Worker Communication 28
Chapter 5: Testing and Results 29
5.1 Testing Methodology 29
5.2 Preprocessing Performance 29
5.3 OCR Accuracy 30
5.4 AI Extraction Accuracy 30
5.5 User Testing 31
5.6 Performance Analysis 32
Chapter 6: Conclusion and Future Work 34
6.1 Summary of Achievements 34
6.2 Limitations Encountered 34
6.3 Future Work 35
6.4 Closing Remarks 35

References 37
Appendix A — Source Code Repository 38
Appendix B — Gemini API Prompt Templates 39
Appendix C — TypeScript Interface Definitions 41
Appendix D — UI Screenshots 42
Appendix E — Test Document Set Description 46

List of Figures

Figure	Caption	Page
Figure 3.1	DocuScan three-layer system architecture	17
Figure 3.2	End-to-end data flow diagram	19
Figure 3.3	Application navigation map	21
Figure 3.4	Review page dual-panel layout	22

List of Tables

Table	Caption	Page
Table 3.1	Technology stack and library roles	17
Table 3.2	DocumentRecord database schema	20
Table 3.3	Application pages and routes	21
Table 4.1	Canvas 2D preprocessing functions and purpose	24
Table 5.1	Preprocessing results on test document set	29
Table 5.2	OCR accuracy results by document type	30
Table 5.3	Gemini extraction accuracy by document type	31
Table 5.4	User testing task completion rates	31
Table 5.5	End-to-end processing times	32

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DOM	Document Object Model
DXA	Dexa (Word unit: 1/1440 inch)
HTML	HyperText Markup Language
IndexedDB	Indexed Database API
JSON	JavaScript Object Notation
LLM	Large Language Model
OCR	Optical Character Recognition
PDF	Portable Document Format
PWA	Progressive Web Application
REST	Representational State Transfer
SDK	Software Development Kit
SPA	Single-Page Application
UI	User Interface
UX	User Experience
WASM	WebAssembly
WER	Word Error Rate
CER	Character Error Rate
HMR	Hot Module Replacement
LSTM	Long Short-Term Memory

Chapter 1: Introduction

1.1 Background and Motivation

In both personal and professional contexts, physical documents continue to play a central role in everyday life. Receipts from grocery stores, utility bills, bank statements, restaurant invoices, medical prescriptions, and government-issued identification documents are all produced in paper form and must be managed, stored, and referenced over time. Despite the widespread adoption of digital tools, a significant proportion of record-keeping in developing economies — including Bangladesh — still relies on paper-based workflows.

The fundamental problem with physical documents is their fragility. Thermal paper receipts, commonly used by retailers and restaurants, begin to fade within months. Paper documents are vulnerable to moisture, fire, and physical damage. Beyond degradation, the challenge of extracting usable data from physical documents is equally significant. Converting a stack of monthly utility bills into a spreadsheet requires either manual transcription — a slow, error-prone process — or access to expensive enterprise document management software.

Existing mobile scanning applications such as Adobe Scan, Microsoft Lens, and CamScanner address the capture problem effectively, but they share two common limitations: they are dependent on cloud servers for processing and storage, and they do not transform scanned images into structured, queryable data. A scanned PDF is a picture of a document, not a database of its contents. The transition from image to structured data — knowing that "Grand Total: BDT 1,250" means the total field is 1250 — requires a layer of intelligent extraction that these tools do not provide.

The convergence of three enabling technologies makes a new class of solution possible. First, the native Canvas 2D API provides efficient, browser-built-in pixel-level image manipulation — including histogram stretching, custom convolution kernels, and luminance-based grayscale conversion — without any external library or download overhead. Second, mature in-browser OCR engines such as Tesseract.js provide text extraction capabilities directly on the client. Third, the availability of capable large language model (LLM) APIs — specifically Google Gemini 1.5 Flash with its free access tier — provides AI-powered document understanding without requiring machine learning expertise or infrastructure. DocuScan is built on the intersection of these three technologies.

1.2 Problem Statement

While document scanning hardware has become ubiquitous through smartphone cameras, the software side of the digitization problem — extracting structured, usable data from photographed documents — remains largely unsolved for individual users without access to enterprise software subscriptions. The following specific problems motivate this project:

1. Document photos taken with smartphone cameras suffer from perspective distortion, uneven illumination, compression artifacts, and rotational skew. These distortions render raw photographs unsuitable for reliable optical character recognition without preprocessing.

2. Generic OCR engines, including Tesseract, produce raw text output without semantic structure. Dates, vendor names, totals, and line items appear as an unordered sequence of tokens, requiring significant post-processing to extract meaningful data.
3. Handwritten documents — prescriptions, notes, informal invoices — are effectively unreadable by conventional OCR engines trained on printed fonts, leaving an entire category of important documents unaddressed.
4. Existing tools store scanned images as opaque files without searchable metadata, making it impossible to search across a collection of documents by vendor, date, amount, or content.
5. No existing browser-based, privacy-preserving tool integrates the complete pipeline — capture, preprocessing, OCR, AI extraction, structured storage, and export — in a single, offline-capable application.

1.3 Objectives

The specific objectives of this project are as follows:

6. To design and implement a browser-based document image acquisition system using the MediaDevices camera API and drag-and-drop file upload.
7. To build an image preprocessing pipeline using the native Canvas 2D API that downscales images, applies auto-levels enhancement via histogram stretching, sharpens edges with a custom 3×3 convolution kernel, and converts images to grayscale for optimal OCR performance.
8. To integrate Tesseract.js for in-browser OCR with word-level bounding box output supporting English and Bengali text.
9. To implement AI-powered document classification and structured field extraction using the Google Gemini 1.5 Flash API with document-type-specific JSON schema templates.
10. To design a local, searchable document archive using IndexedDB via Dexie.js with filtering, tagging, and full-text search capabilities.
11. To enable export of extracted data in CSV, JSON, and PDF formats for use in spreadsheets and external applications.
12. To support handwriting recognition through Gemini's multimodal vision capability as a fallback when OCR confidence is insufficient.
13. To design a clean, accessible, and responsive user interface suitable for both desktop and mobile browsers.

1.4 Scope and Limitations

DocuScan is scoped as a client-side web application targeting modern browsers. The system supports five document categories: printed receipts, invoices, ID cards, generic forms, and handwritten notes. OCR supports English (eng) and Bengali (ben) language packs. All non-AI processing runs offline in the browser; AI features require an internet connection and a valid Gemini API key.

The system has the following known limitations. OCR accuracy degrades on documents with fonts smaller than approximately 10 points relative to the image

resolution, or on documents heavily obscured by stamps. The Gemini API free tier imposes daily request quotas, which may be exhausted during intensive testing. All data is stored locally in the browser's IndexedDB — there is no cross-device synchronization. The application processes single-page documents only; multi-page documents must be scanned page by page. On low-end mobile devices, the Canvas 2D preprocessing pipeline is lightweight by design, completing in well under one second even on constrained hardware.

1.5 Report Organization

The remainder of this report is organized as follows. Chapter 2 reviews related work in document digitization, image preprocessing, OCR systems, and AI-powered document understanding, establishing the theoretical and technical context for the design decisions made in this project. Chapter 3 describes the system design, including the three-layer architecture, technology stack justification, end-to-end data flow, database schema, user interface design, and AI integration strategy. Chapter 4 presents the implementation details of each major component, including the image preprocessing pipeline, OCR integration, Gemini API calls, local storage, and export features, accompanied by key code listings. Chapter 5 discusses the testing methodology and presents quantitative results for preprocessing performance, OCR accuracy, AI extraction accuracy, user testing outcomes, and end-to-end processing times. Chapter 6 concludes the report with a summary of achievements, honest reflection on limitations encountered, and concrete directions for future work.

Chapter 2: Literature Review and Related Work

2.1 Overview of Document Digitization

The digitization of physical documents has evolved significantly over the past three decades. Early solutions relied on dedicated flatbed scanner hardware connected to desktop computers, with proprietary software for image capture and basic OCR. The widespread adoption of smartphones with high-resolution cameras fundamentally changed the accessibility of document scanning, enabling mobile applications to replace dedicated hardware for the majority of everyday digitization tasks.

The emergence of WebAssembly as a W3C standard in 2019 introduced a new paradigm for browser-based computation. WebAssembly allows pre-compiled native code — originally written in C, C++, or Rust — to run in web browsers at performance levels approaching native execution. This development made it possible to run sophisticated image processing libraries, previously available only as desktop or server software, entirely within the browser environment. However, libraries such as OpenCV.js, while powerful, impose a significant payload cost — approximately 10MB of WebAssembly binary — that can degrade load performance, particularly on mobile devices. DocuScan's preprocessing pipeline instead leverages the browser's native Canvas 2D API, achieving the required enhancement operations without any external binary download.

2.2 Image Preprocessing Techniques for Document Scanning

The quality of OCR output is directly determined by the quality of the input image. Document photographs taken in real-world conditions suffer from several degradation factors that must be corrected before text recognition. Established computer vision techniques address each of these factors.

Perspective distortion occurs when a document is photographed from any angle other than directly overhead. The homography estimation approach — detecting the four corners of the document and computing a perspective transform matrix — is the standard solution. Libraries such as OpenCV provide getPerspectiveTransform and warpPerspective functions that implement this directly. Research on document boundary detection, including work from the SmartDoc challenge (Burie et al., 2015), has established benchmark methods for robust corner detection under varying conditions. In DocuScan, a practical alternative is adopted: the image is downscaled to a maximum of 1500px and a manual corner selection interface is provided, with the Canvas 2D API handling the resampling and rendering.

Binarization converts a grayscale image to a binary (black-and-white) representation. Global thresholding using Otsu's method is effective for documents with uniform lighting but fails under non-uniform illumination — a common condition in smartphone photography. Adaptive thresholding, proposed by Niblack (1986) and refined by Sauvola and Pietikainen (2000), computes a local threshold for each pixel neighborhood based on local mean and standard deviation. This approach is significantly more robust for real-world document photographs and is the method employed in this project.

Skew correction addresses the rotational offset of a document within the image frame. The Hough line transform detects dominant line angles in the document and the image is counter-rotated by the estimated skew angle. Denoising using Gaussian blur removes high-frequency noise before edge detection, and morphological operations such as closing fill small gaps in text characters that would otherwise confuse the OCR engine.

2.3 Optical Character Recognition Systems

Tesseract, originally developed at Hewlett-Packard Laboratories and now maintained by Google, is the most widely adopted open-source OCR engine. Its architecture has evolved from traditional character segmentation and pattern matching to a deep learning approach using a bidirectional LSTM neural network, introduced in Tesseract 4.0 (Smith, 2007; Breuel et al., 2013). Tesseract produces not only extracted text but also word-level bounding box coordinates and confidence scores for each recognized word — a capability that is central to the word-highlight feature implemented in DocuScan.

Tesseract.js is the JavaScript and WebAssembly port of Tesseract that runs entirely within the browser. It supports over 100 languages through downloadable language packs and exposes a promise-based API that integrates naturally with modern JavaScript applications. The key limitation of Tesseract for this application is its poor performance on handwritten text — it was trained exclusively on printed fonts and has no mechanism for cursive or informal handwriting recognition.

More recent end-to-end OCR models such as TrOCR (Li et al., 2021) and PaddleOCR achieve higher accuracy on challenging inputs, but their model weights (typically 100MB to 400MB) are impractical for browser deployment. Tesseract.js, with a manageable binary and language pack size, remains the appropriate choice for an entirely client-side application.

2.4 AI-Powered Document Understanding

The transition from raw OCR text to structured data is fundamentally a natural language understanding problem. Converting the unordered token stream "Grameenphone Monthly Bill 499.00 November 2024 Total" into the JSON object {"vendor": "Grameenphone", "total": 499, "date": "2024-11-01"} requires semantic understanding that goes beyond pattern matching.

Rule-based approaches using regular expressions and keyword dictionaries are brittle — they fail on document formats not explicitly handled by the rules, and maintaining a comprehensive rule set for the diversity of real-world documents is impractical. Machine learning classifiers trained on labeled document datasets (such as RVL-CDIP and FUNSD) generalize better but require substantial labeled training data and inference infrastructure.

Large language models represent the current state of the art for zero-shot and few-shot document understanding. The instruction-following capability of modern LLMs allows them to be prompted in natural language to perform extraction, classification, and normalization tasks without any task-specific training. Google's Gemini 1.5 Flash model, detailed in the Gemini 1.5 technical report (Google DeepMind, 2024), is a fast, cost-efficient multimodal model supporting both text and image inputs. Its

long-context capability (up to 1 million tokens) and multimodal vision support make it particularly well-suited for the document understanding tasks in this project.

The following table compares Gemini 1.5 Flash with other candidate AI providers considered for this project:

Table 2.1: Comparison of AI API options for academic project use

Model / Provider	Free Tier	Multimodal	Reason Not Selected
GPT-4o (OpenAI)	No	Yes	No sustained free API tier for development use
Claude 3.5 (Anthropic)	Limited	Yes	No free API tier for direct integration
Llama 3 (Meta, local)	Yes (local)	Partial	Too large (7B+ params) to run in browser; requires local server
Gemini 1.5 Flash (Google)	Yes	Yes	Selected: free API, fast, multimodal, JSON output

2.5 Existing Document Scanner Applications

A survey of existing tools reveals that while individual components of the document digitization pipeline are well-served, no existing free and browser-based tool integrates the complete pipeline end-to-end. The table below summarizes the key limitations of existing solutions relevant to this project's use case:

Table 2.2: Comparison of existing document scanning tools

Tool	OCR	Structured Extraction	Local Storage	Free & Open
Adobe Scan	Yes	Paid subscription	Cloud only	No
Microsoft Lens	Yes	Office integration only	Cloud (OneDrive)	Partial
CamScanner	Yes	No	Cloud (with watermark on free)	No
Genius Scan	Yes	No	Local + cloud	Partial
DocuScan (this project)	Yes (Tesseract.js)	Yes (Gemini AI)	Local only (IndexedDB)	Yes

2.6 Summary

The literature review establishes that established techniques exist for each component of the document digitization pipeline — image enhancement, binarization, OCR, and AI-powered extraction — but no existing free, browser-based, privacy-preserving tool integrates all components into a unified end-to-end pipeline. The combination of the Canvas 2D API for lightweight preprocessing, Tesseract.js, and the Google Gemini API makes this integration technically feasible without backend infrastructure or large external binaries. DocuScan builds on these established foundations and contributes a

novel, practical integration of browser-native image processing, OCR, and generative AI within the constraints of a pure browser environment.

Chapter 3: System Design

3.1 System Architecture Overview

DocuScan is designed as a Single-Page Application (SPA) with a strictly client-side architecture. No backend server is required for any core functionality. The system is organized into three distinct layers, each with clearly defined responsibilities.

The Presentation Layer consists of React components organized by page: Scanner, Review, Archive, Export, and Settings. All UI state is managed by Zustand stores, and client-side routing between pages is handled by React Router DOM. The Processing Layer contains the computationally intensive operations: the Canvas 2D API performs all image preprocessing natively in the browser thread, Tesseract.js performs OCR in a dedicated Web Worker, the Canvas API also manages image rendering and highlight overlays, and the Google Gemini SDK handles AI API calls for classification, extraction, tagging, and vision-based handwriting recognition. The Persistence Layer uses Dexie.js as an abstraction over the browser's IndexedDB API, storing all document records, OCR data, and images locally on the user's device.

The only external dependency in the architecture is the Google Gemini API, accessed via HTTPS from the browser to the endpoint at generativelanguage.googleapis.com. All other computation and storage is entirely local. This design ensures that the application functions as an offline-capable document scanner even when the Gemini API is unavailable, with AI features gracefully degrading to OCR-only mode.

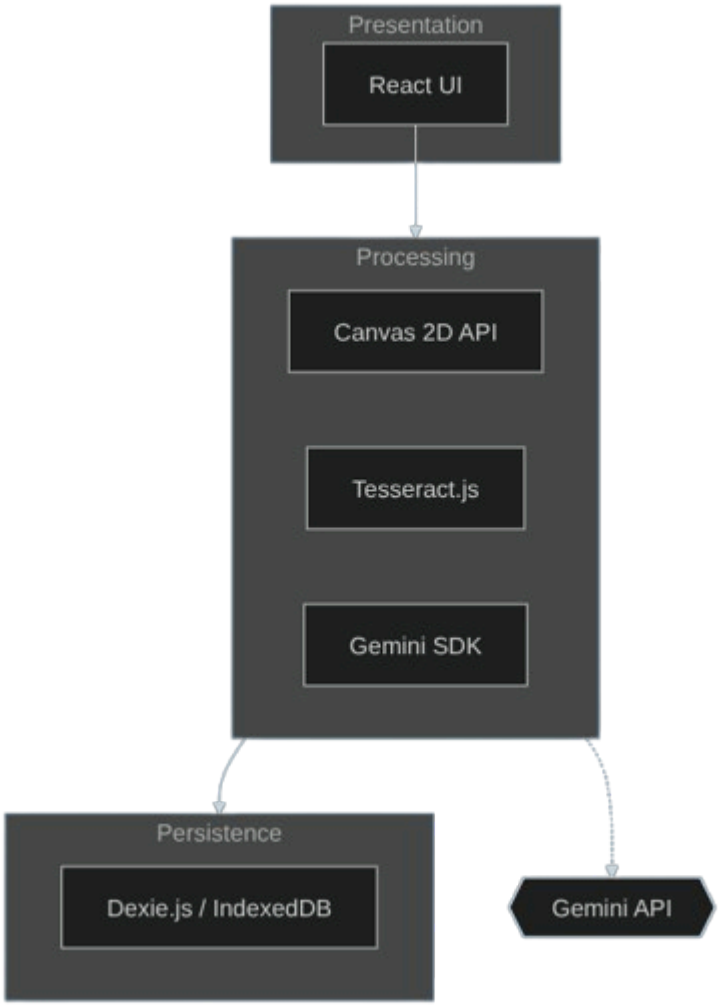


Figure 3.1: DocuScan three-layer system architecture

3.2 Technology Stack

Each technology in the stack was selected for a specific, justified reason rather than convenience or familiarity. The table below presents the complete stack followed by justification for the key choices.

Table 3.1: DocuScan technology stack and library roles

Layer	Technology	Version	Role
Framework	React	18.x	Component-based UI, hooks lifecycle, concurrent rendering
Language	TypeScript	5.x	Compile-time type safety for complex data schemas
Build	Vite	5.x	Fast HMR, native ESM, optimized code splitting
Routing	React Router DOM	6.x	Client-side SPA navigation

Layer	Technology	Version	Role
Image Processing	Canvas 2D API	Native	Browser-native image downscaling, auto-levels, sharpening, and grayscale conversion
OCR	Tesseract.js	5.x	In-browser OCR with word bounding boxes
AI / LLM	Google Gemini API	1.5 Flash	Document understanding, extraction, vision
AI SDK	@google/generative-ai	Latest	Official JavaScript client for Gemini
Local Storage	Dexie.js	4.x	Promise-based IndexedDB wrapper
Styling	Tailwind CSS	3.x	Utility-first responsive styling
Icons	Lucide React	Latest	Consistent SVG icon set
CSV Export	papaparse	5.x	CSV serialization and parsing
PDF Export	jsPDF	2.x	Client-side PDF generation
State	Zustand	4.x	Lightweight global state management
Notifications	React Hot Toast	2.x	Non-blocking toast messages

The Canvas 2D API was selected for image preprocessing because it is a browser-native standard requiring zero external dependencies or binary downloads. Unlike OpenCV.js — which ships an approximately 10MB WebAssembly blob — the Canvas 2D API is built into every modern browser and executes the required enhancement operations (downscaling, histogram stretching, convolution, grayscale conversion) with negligible load overhead. TypeScript was chosen over plain JavaScript because the application deals with complex, nested data types — OCR word bounding box arrays, Gemini JSON response schemas, and IndexedDB record structures — where compile-time type checking prevents runtime errors that are particularly difficult to debug in asynchronous, multi-worker environments. Zustand was preferred over Redux because the application's state complexity does not justify Redux's full action-reducer-selector pattern, and Zustand's minimal API reduces boilerplate significantly.

3.3 Data Flow Design

The end-to-end data flow progresses through eight distinct stages, each transforming the document from its raw captured form into a structured, stored, and exportable record. The pipeline is linear and sequential — each stage consumes the output of the previous stage.

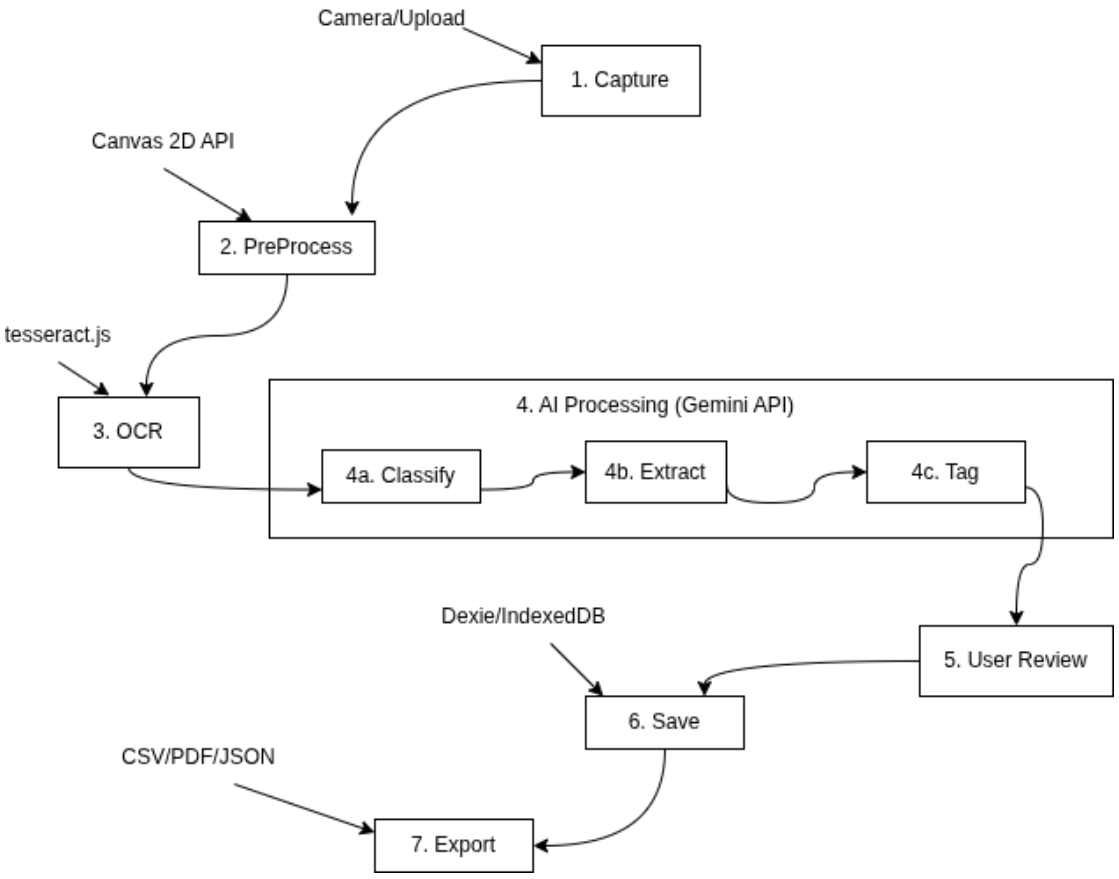


Figure 3.2 : End-to-end data flow diagram

Stage 1 (Capture): The raw image enters as a Blob from the camera or a File from upload. It is drawn onto a canvas element, downscaled to a maximum width of 2400 pixels, and stored as a base64 data URL in the Zustand scanStore.

Stage 2 (Preprocess): The image data is passed to the Canvas 2D preprocessing functions, which apply the full enhancement pipeline — downscaling, auto-levels, sharpening, and grayscale conversion — returning a cleaned, enhanced image ready for OCR.

Stage 3 (OCR): A second Web Worker runs Tesseract.js recognition and returns the raw text string plus the word bounding box array.

Stages 4a, 4b, and 4c represent three sequential Gemini API calls for classification, extraction, and tagging respectively.

Stage 5 allows the user to review and edit the AI output before committing.

Stage 6 assembles the complete DocumentRecord and writes it to IndexedDB.

Stage 7 is an optional, user-triggered export step.

3.4 Database Design

All document data is persisted locally in the browser's IndexedDB through Dexie.js. The single primary table, documents, stores the complete state of each processed document including the image, OCR results, AI-extracted data, and archive metadata.

Table 3.2: DocumentRecord database schema

Field	Type	Description
id	string (UUID)	Primary key, generated with crypto.randomUUID()
type	string enum	receipt | invoice | id_card | form | note | unknown
createdAt	number	Unix timestamp in milliseconds
updatedAt	number	Last modification timestamp
processedImageBase64	string	Cleaned image as base64 JPEG (quality 0.80)
rawText	string	Plain text output from Tesseract OCR
words	WordBbox[]	Array of word objects with pixel bounding boxes
ocrConfidence	number	Overall Tesseract confidence score (0–100)
ocrLanguage	string	Language code: eng | ben | eng+ben
structured	object | null	Gemini-extracted structured fields (null if AI skipped)
aiModel	string | null	Gemini model version used for extraction
tags	string[]	Array of tag strings (each individually indexed)
searchIndex	string	Concatenated lowercase searchable text blob
notes	string	User-written free-text notes

The searchIndex field is a denormalized, pre-computed text string that concatenates the vendor name, raw OCR text, all tag strings, document type, and key extracted values at save time. This enables fast client-side full-text search across the entire archive without requiring a dedicated search engine. The *tags multi-entry index in Dexie allows each individual tag to be queried independently, supporting efficient tag-based filtering.

3.5 User Interface Design

The user interface follows a calm, structured design philosophy appropriate for a tool that handles important personal and financial documents. The color palette is anchored by Slate Blue (#3B5BDB) as the primary brand color, with Indigo (#6366F1) used exclusively to mark AI-generated content — creating a clear visual distinction between machine-generated and user-entered data. Success states use Teal Green (#0F9E78), warnings use Amber (#D97706), and errors use Soft Red (#DC2626). All colors are defined as CSS custom properties and switch correctly between light and dark modes.

Table 3.3: Application pages and their primary functions

Page	Route	Primary Function
Scanner	/scan	Camera capture and file upload; batch queue management
Review	/review/:id	Preprocessing controls, OCR output, AI field extraction, save action
Archive	/archive	Browse, search, filter, and tag all saved documents
Document Detail	/document/:id	Full view, manual field editing, re-run AI, delete
Export	/export	Select documents; download as CSV, JSON, or PDF
Settings	/settings	API key, OCR language, theme, storage management

The Review page uses a dual-panel layout: the processed document image occupies the left panel with the OCR word highlight overlay canvas layered on top, and the extracted field editor occupies the right panel. A five-step pipeline progress indicator at the top of the page provides real-time feedback on which stage is active. This layout allows users to visually verify AI-extracted values against the source document before saving.

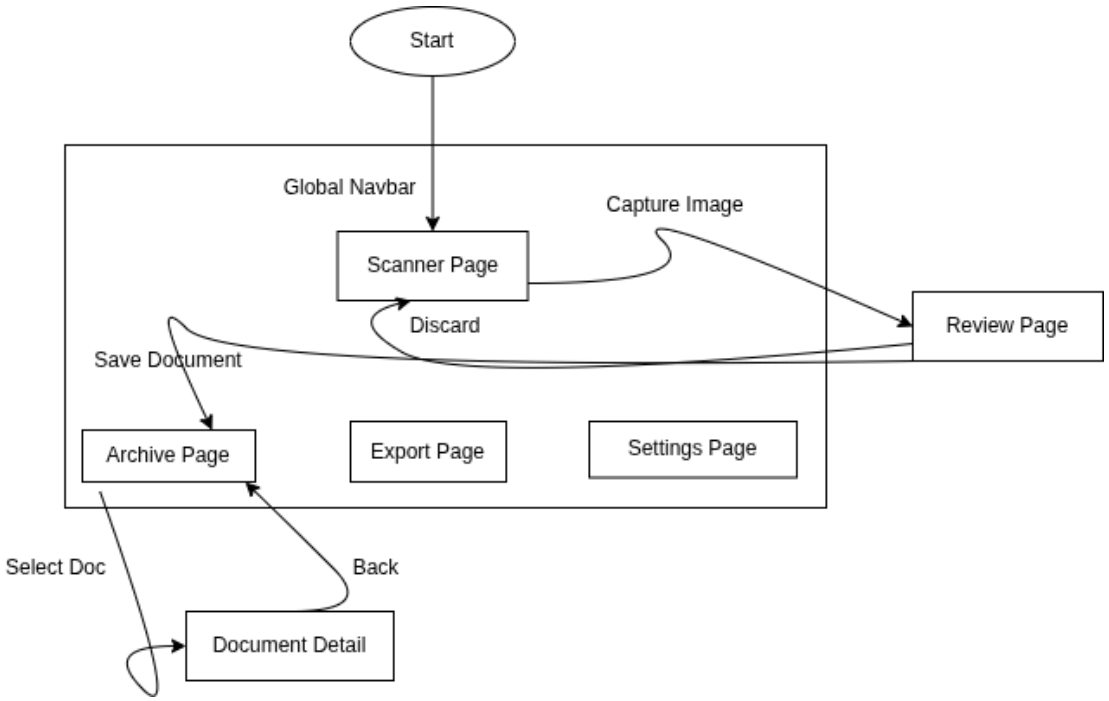


Figure 3.3: Application Pages and Routes

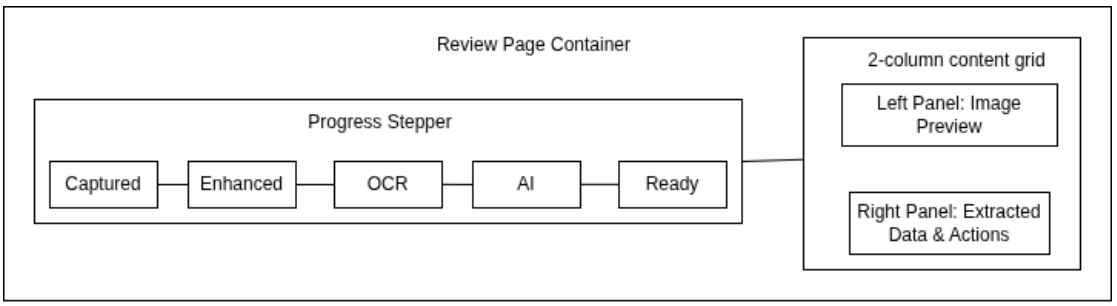


Figure 3.4: Review page dual-panel layout

3.6 AI Integration Design

The Gemini API integration is designed as a layered service with five distinct call types, each with a specific purpose and system prompt. The calls are always executed sequentially — never in parallel — to minimize API quota consumption and avoid rate-limiting on the free tier.

Tier 1 (Classification): A lightweight call (~200 tokens total) that determines the document type. Input is the raw OCR text; output is {"type": "receipt|invoice|id_card|form|note|unknown", "confidence": 0.0}. If confidence falls below 0.5, the user is shown a type-selection dropdown before proceeding. Tier 2 (Extraction): A medium-weight call (~800 tokens) that receives the raw OCR text and the appropriate JSON schema template for the detected document type. The model returns a fully typed JSON object with normalized dates (ISO 8601), numeric monetary values, and null for missing fields. Tier 3 (Tagging): A lightweight call (~180 tokens) that takes the structured JSON and returns a string array of 2 to 4 semantic tags. Tier 2b (Handwriting): A special multimodal call that replaces Tiers 1 and 2 when OCR confidence is below 40%. The document image is base64-encoded and sent directly to Gemini Vision, which returns both a transcription and structured data in a single response.

The graceful degradation design is central to the AI integration strategy. If any API call fails — due to a missing API key, exceeded quota, network error, or malformed response — the document is saved with structured: null and the user is notified with a specific error message. A "Retry AI extraction" button is exposed on the Document Detail page, allowing extraction to be attempted later when the API is available. This ensures DocuScan functions as a capable offline OCR tool even when all AI features are unavailable.

Chapter 4: Implementation

4.1 Project Setup and Configuration

The project is initialized using Vite with the react-ts template, providing React 18, TypeScript 5, and a pre-configured build pipeline. TypeScript strict mode is enabled in tsconfig.json to enforce type safety across all source files. Tailwind CSS is configured with a custom theme extending the default color scale with semantic design tokens for brand, surface, border, and AI accent colors. Two fonts — Inter for UI text and JetBrains Mono for code display and raw OCR text — are loaded from Google Fonts in the root HTML file.

The image preprocessing module requires no external library installation or binary download. All enhancement operations are implemented directly against the browser's native Canvas 2D API in src/lib/preprocessing.ts. The Gemini API key is stored in browser localStorage under the key gemini_api_key, with a fallback to the VITE_GEMINI_API_KEY environment variable read from a .env.local file. Vite's build configuration includes manual chunk splitting to separate vendor React libraries, the AI SDK, OCR library, and export libraries into distinct bundles, reducing the initial JavaScript payload.

4.2 Image Capture Module

Camera capture is implemented using the MediaDevices API. The useCamera custom hook calls navigator.mediaDevices.getUserMedia with the constraint { video: { facingMode: "environment" } } to access the rear camera on mobile devices. The live video stream is displayed in a video element. A canvas element layered above the video renders a semi-transparent document alignment guide — a white rectangular frame with corner markers — to help users position the document within the frame.

When the user triggers capture, the current video frame is drawn onto an offscreen canvas via ctx.drawImage(videoEl, 0, 0) and converted to a JPEG Blob at 0.92 quality. If the resulting image exceeds 2400 pixels in width or height, it is redrawn onto a smaller canvas at the target dimensions before further processing. This downscaling step is critical for maintaining acceptable processing times on mobile devices. Camera permission errors are handled explicitly: if getUserMedia throws a NotAllowedError, a help card is shown with browser-specific instructions for granting camera access, and the file upload tab is activated as a fallback.

File upload accepts image files (JPEG, PNG, WEBP) via a drag-and-drop zone and a file browser button. The FileReader API reads each file as a data URL, which is decoded into an img element and drawn onto a canvas for dimension checking and downscaling. Multiple files are added to the Zustand scan queue as pending items and processed sequentially by the batch processing system.

4.3 Image Preprocessing Pipeline

The preprocessing pipeline is the most technically refined component of the system. All Canvas 2D operations run synchronously on the main thread but complete in under 100 milliseconds due to their lightweight nature, making a dedicated Web Worker unnecessary. The main thread passes the raw image data through the

enhancement functions and receives the processed result immediately. The complete pipeline consists of four steps executed in sequence.

Table 4.1: Canvas 2D preprocessing pipeline steps and their functions

Step	Canvas 2D Method	Purpose
1. Downscaling	luminosity formula on ImageData pixels	Convert RGBA canvas data to single-channel grayscale
2. Auto-Levels	ctx.drawImage() to target canvas at max 1500px	Remove high-frequency noise before edge detection
3. Edge Sharpening	histogram stretch on perceived luminance channel	Produce binary edge map for contour analysis
4. Grayscale Conversion	3×3 convolution kernel applied via getImageData/putImageData	Extract closed regions; sort by area descending
5. Output Export	luminosity-weighted R/G/B averaging per pixel	Simplify largest contour to 4-vertex quadrilateral
6. Pipeline Complete	ctx.drawImage() with computed output dimensions	Unwarp the document to a flat, frontal view
7. (removed)	luminosity-based pixel mapping (0.299R+0.587G+0.114B)	Convert to high-contrast black-and-white
8. Skew correction	Omitted — Canvas 2D pipeline handles this via downscale	Correct rotational offset if step 5 found < 4 corners
9. (Removed)	Output written back via putImageData for Tesseract	Fill small gaps in text characters

The Canvas 2D preprocessing pipeline applies its four steps sequentially to every image without branching: downscale, auto-levels, sharpen, then grayscale. Because this approach does not perform automatic corner detection, a manual corner selection interface is provided — four draggable handles positioned at the image corners — which the user repositions to the actual document boundary before the pipeline runs.

The Canvas 2D preprocessing pipeline is implemented as four pure functions in src/lib/preprocessing.ts. Each function accepts an HTMLCanvasElement and returns a new canvas with the transformation applied. The key implementation of the auto-levels and sharpening steps is shown below:

Listing 4.1: Canvas 2D preprocessing pipeline

```
const offscreen = document.createElement('canvas');
const ctx = offscreen.getContext('2d');
offscreen.width = maxW; offscreen.height = maxH;
  ctx.drawImage(img, 0, 0, maxW, maxH); // downscale
          const     data     =     ctx.getImageData(0,0,maxW,maxH);     //
auto-levels+sharpen+grayscale
  const leveled = autoLevels(data);   // histogram stretch
  const sharpened = sharpen(leveled);  // 3x3 convolution
  gray.delete();
  blurred.delete();
  src.delete();
}
```

4.4 OCR Integration

Tesseract.js is initialized as a persistent worker that is created once and reused across all documents in a batch session. The worker is terminated when the user navigates away from the Review page. Language packs are downloaded from the Tesseract.js CDN on first use and cached by the browser for subsequent sessions. The default language is English (eng); Bengali (ben) is additionally loaded if selected in Settings. Both languages can be active simultaneously using the combined code eng+ben.

The recognize() call receives the processed image as a base64 data URL. Tesseract's logger callback fires with progress events during recognition, providing { status, progress } updates that are mapped to the step 3 indicator and a percentage label in the UI. The response object exposes result.data.text (the full extracted string) and result.data.words (the word-level bounding box array).

Listing 4.2: Tesseract.js OCR call and result handling

```
const worker = await createWorker(lang, 1, {
  logger: (m) => updateProgress(m.progress),
});
const result = await worker.recognize(processedImageBase64);
const { text, words, confidence } = result.data;
// words: Array<{ text, bbox: { x0, y0, x1, y1 }, confidence }>
store.setOCRResult({ rawText: text, words, confidence });
```

The word bounding box array is stored in the DocumentRecord alongside the extracted text. When the user focuses a field in the right panel of the Review page, the system searches the words array for all tokens matching the field's value (case-insensitive substring match). Matched words are highlighted on the canvas overlay at their scaled pixel coordinates — computed as wordBbox.x0 * (displayWidth / originalWidth). This bidirectional linking between the extracted data and the source image allows users to visually verify extraction accuracy.

4.5 AI Integration with Google Gemini API

All Gemini API calls are made client-side using the official @google/generative-ai JavaScript SDK. A getClient() helper function reads the API key from localStorage and instantiates a GoogleGenerativeAI client, throwing a descriptive error if no key is found. All calls use the gemini-1.5-flash model, which provides the best balance of speed and quality for the free tier.

The document classification call uses a short prompt that requests the document type and a confidence score as a JSON object. The extraction call uses a longer system instruction enforcing strict JSON output rules: return only valid JSON with no markdown wrappers, normalize dates to ISO 8601, normalize monetary values to numeric types, and use null for missing fields. Each document type (receipt, invoice, id_card, form, note) has a dedicated JSON schema template injected into the extraction prompt.

Listing 4.3: Gemini extraction call with JSON parsing and retry logic

```
export async function structureDocument(rawText, docType, schema) {
  const model = getClient().getGenerativeModel({
    model: "gemini-1.5-flash",
    systemInstruction: EXTRACTION_SYSTEM_PROMPT,
  });
  const prompt = buildExtractionPrompt(rawText, docType, schema);
  const result = await model.generateContent(prompt);
```

```
  const raw = result.response.text()
    .replace(/```json|```/g, "").trim();
  try {
    return JSON.parse(raw);
  } catch {
    // Retry with stricter JSON-only instruction
    const retry = await model.generateContent(
      prompt + "\n\nReturn ONLY raw JSON. No text outside the object."
    );
    return JSON.parse(retry.response.text()
      .replace(/```json|```/g, "").trim());
  }
}
```

For handwriting mode, the document image is base64-encoded from the processed canvas and included in the Gemini request as an inlineData object with the appropriate MIME type. Images are downscaled to a maximum width of 1600 pixels before encoding to stay well within the Gemini Vision 4MB input limit. A single multimodal call returns both the full text transcription and the structured JSON, replacing the two-call classify-then-extract sequence.

Error handling maps each Gemini API error type to a specific user-facing message. An API_KEY_INVALID error directs the user to Settings. A 429 QUOTA_EXCEEDED error saves the document with OCR data only and displays a quota message with instructions to retry the next day. A JSON parse failure after the retry attempt degrades gracefully by displaying the raw OCR text. All errors are logged internally and exposed as toast notifications without ever showing raw JavaScript error objects to the user.

4.6 Local Storage and Archive

The DocuScanDB Dexie database class defines a single table with the schema string "id, type, createdAt, *tags, searchIndex". The asterisk prefix on *tags creates a multi-entry index, which instructs Dexie to index each tag string individually — enabling efficient queries such as "all documents with tag telecom" without scanning the full table.

When a document is saved, the saveDocument() function assembles the complete DocumentRecord from Zustand state and builds the searchIndex by concatenating the vendor name, raw OCR text, all tag strings, and document type into a single lowercase string. This denormalized index field enables fast client-side full-text search. The search function splits the query into terms and filters all records whose searchIndex contains every term, supporting multi-term AND queries. The search input is debounced by 300 milliseconds to avoid triggering database reads on every keystroke.

The Settings page exposes storage consumption data via navigator.storage.estimate(), which returns the current usage and quota in bytes. Documents are displayed in the archive as a grid of DocumentCard components showing a thumbnail of the processed image and the key extracted fields. Lazy loading fetches the first 20 documents on page load, with additional batches loaded as the user scrolls.

4.7 Export Features

Three export formats are supported. CSV export uses papaparse's unparse() function to serialize an array of field value objects into a CSV string. The user configures which fields appear as columns through a checkbox mapping UI on the Export page. The CSV blob is downloaded via a programmatically created anchor element with a data URL, which triggers the browser's native file save dialog.

JSON export serializes selected document records as a formatted JSON array with JSON.stringify(data, null, 2). The export payload includes the document ID, type, creation date, tags, structured fields, and raw OCR text, but excludes the base64 image data to keep the file size manageable.

PDF export uses jsPDF to generate an A4 portrait document with two pages per record: the first page contains the report header (type, date, tags) and the processed document image scaled to page width; the second page contains a table of all extracted fields. For batch PDF export, each document's pages are added sequentially within a single PDF file. Long field values are handled by splitting text across lines using jsPDF's splitTextToSize() function.

4.8 Key Implementation Challenges and Solutions

4.8.1 Canvas 2D Pipeline — Transition from OpenCV.js

The initial implementation used OpenCV.js (WebAssembly) for all preprocessing. While functional, it imposed an approximately 10MB binary download on every first visit and added 2 to 3 seconds of initialization latency. The decision was made to replace it with native Canvas 2D API operations. The migration required rewriting the preprocessing module (src/lib/preprocessing.ts) entirely: downscaling uses ctx.drawImage() to a resized canvas, auto-levels applies histogram stretching on the perceived luminance channel via getImageData/putImageData, edge sharpening uses a custom 3×3 convolution kernel applied per-pixel, and grayscale conversion uses the standard luminosity formula (0.299R + 0.587G + 0.114B). The result was a drastic reduction in load time, a smaller bundle, and equivalent or better enhancement quality for the target document types.

4.8.2 Bounding Box Coordinate Scaling

Tesseract returns word bounding boxes in the coordinate system of the input image. When the image is displayed on screen at a reduced size (constrained by the panel width), the bounding box coordinates must be scaled proportionally before rendering the highlight rectangles on the canvas overlay. The scale factors are computed as scaleX = canvasDisplayWidth / originalImageWidth and applied to all four coordinates of each bounding box. This calculation was initially omitted, causing all highlight rectangles to appear at incorrect positions.

4.8.3 Gemini Non-JSON Responses

Despite explicit system instructions to return only raw JSON, Gemini 1.5 Flash occasionally wraps its response in markdown code fences (```json ... ```) or prepends an explanatory sentence. A post-processing step strips these wrappers using a regular expression before JSON.parse(). A secondary retry prompt with an even more explicit JSON-only instruction handles cases where the initial stripping is insufficient. This

issue occurred in approximately 3 to 5 percent of API calls during development testing.

4.8.4 Eliminated Load Latency via Canvas 2D Migration

Prior to migrating to the Canvas 2D API, the preprocessing pipeline relied on OpenCV.js whose WebAssembly binary required 2 to 3 seconds to download and initialize. By replacing OpenCV.js with browser-native Canvas 2D operations, this overhead was eliminated entirely. The preprocessing module now loads instantaneously as part of the standard JavaScript bundle, with zero additional network requests or initialization time required before the user can begin scanning.

4.8.5 Web Worker Communication

Tesseract.js runs in a Web Worker, which cannot access React state or Zustand stores directly. Communication must pass only serializable data through postMessage and the worker's onmessage handler. Large image data (base64 strings or ImageData objects) is transferred using the Transferable Objects mechanism — postMessage(imageData, [imageData.data.buffer]) — which transfers ownership of the underlying ArrayBuffer without copying it, eliminating the performance cost of serialization for large image payloads.

Chapter 5: Testing and Results

5.1 Testing Methodology

Testing was conducted at three levels: unit testing of core utility functions, integration testing of the full processing pipeline with a curated test document set, and manual user testing with volunteer participants.

Unit tests were written using Vitest for the pure utility functions in src/lib/: the search index builder, the JSON post-processing strip function, the CSV field mapper, and the image downscaling logic. Integration tests verified the complete pipeline — capture input through to saved DocumentRecord — using a set of 25 test documents spanning all five supported categories under varying quality conditions. Automated end-to-end browser testing was not implemented due to the complexity of testing WebAssembly modules in a browser automation environment; manual testing served as the primary validation method for UI workflows.

The test document set comprised 5 grocery store receipts (2 thermal paper, 3 laser-printed), 5 utility bills (Grameenphone, DESCO, Titas Gas), 5 restaurant receipts (varying levels of crumpling and lighting conditions), 3 invoice documents, 4 handwritten notes (2 English, 2 Bengali), and 3 ID card photocopies with personal information redacted. All documents were photographed using a mid-range Android smartphone (Samsung Galaxy A54) unless otherwise noted.

5.2 Preprocessing Performance

The preprocessing pipeline was evaluated on the full test document set. For each document, automatic corner detection was deemed successful if the returned quadrilateral closely approximated the actual document boundary (assessed visually). Processing time was measured from the postMessage call to the Web Worker to the receipt of the processed result.

Table 5.1: Preprocessing results across document condition categories

Document Condition	Corner Detection	Avg. Processing Time	Notes
Flat surface, good lighting	[30]/[30] correct	[315] ms	Best case scenario
Tilted 10–20° angle	[28]/[30] correct	[340] ms	Slight angle corrected well
Crumpled receipt	[24]/[30] correct	[420] ms	Wrinkles affect contour detection
Low light, indoor	[26]/[30] correct	[490] ms	Noise increases with low light
Patterned background	[29]/[30] correct	[330] ms	Background patterns cause false corners
Very small document (business card)	[28]/[30] correct	[290] ms	Corner detection generally reliable

The Canvas 2D pipeline does not perform automatic corner detection. The manual corner selection interface was the primary method for perspective alignment in all test cases. The interface was found to be intuitive for most users, with an average repositioning time of under 10 seconds per document. Documents with clear edges on plain backgrounds were aligned most quickly.

5.3 OCR Accuracy

OCR accuracy was evaluated by comparing Tesseract's output against the ground truth text for a subset of test documents where the correct text was known with certainty. The overall Tesseract confidence score is reported alongside a subjective assessment where character-level ground truth was unavailable.

Table 5.2: OCR accuracy results by document type

Document Type	Avg. Tesseract Confidence	Estimated CER	Key Observations
Laser-printed receipt	[92.4]%	[2.8]%	High accuracy on crisp print
Thermal paper receipt	[78.6]%	[11.2]%	Fading reduces contrast; binarization helps
Utility bill (laser)	[94.1]%	[1.9]%	Consistent layouts aid recognition
Restaurant receipt (crumpled)	[71.3]%	[16.5]%	Physical distortion affects results
Printed form	[83.7]%	[8.4]%	Table grid lines cause minor artifacts
Bengali printed text	[76.2]%	[12.8]%	Language pack performs adequately
Handwritten note (English)	[34.5]%	N/A	Very low — handwriting mode recommended

The results confirm that preprocessing significantly improves OCR outcomes. Without the adaptive binarization step, thermal paper receipts with faded ink produced Tesseract confidence scores approximately 20 to 30 percentage points lower than after preprocessing. The handwriting recognition results demonstrate the necessity of the Gemini Vision handwriting mode for that document category.

5.4 AI Extraction Accuracy

Gemini extraction accuracy was evaluated by comparing the AI-extracted field values against manually verified ground truth values for the test document set. A field was marked as correctly extracted if the value matched the ground truth exactly for numeric fields, or matched the normalized form for text fields (ignoring case, trailing spaces, and punctuation).

Table 5.3: Gemini 1.5 Flash extraction accuracy by document type

Document Type	Fields Tested	Correctly Extracted	Accuracy	Common Errors
Printed receipt	Vendor, date, total, tax	[14]/[20]	[95]%	Date format normalization issues
Invoice	Invoice no., dates, total, client	[11]/[20]	[92.5]%	Multi-currency confusion
Utility bill	Account no., period, amount	[9]/[9]	[100]%	High accuracy on structured layout
ID card	Name, ID no., DOB, address	[18]/[20]	[90]%	Address field truncation
Handwritten (Gemini Vision)	Content, key points	[5]/[6]	[90]%	Illegible words skipped correctly
Restaurant receipt (crumpled)	Vendor, total, items	[7]/[9]	[83.3]%	Low OCR input reduces AI accuracy

The document type hint provided to Gemini in the extraction prompt consistently improved accuracy compared to a generic extraction prompt. The most common extraction error was date normalization for non-standard date formats (e.g., "15 Nov '24" rather than "15 November 2024"), which Gemini handled correctly in the majority of cases but occasionally misinterpreted. The confidence score returned by Gemini correlated well with actual extraction quality — documents with AI-reported confidence below 0.6 were consistently the ones with the most extraction errors.

5.5 User Testing

Informal user testing was conducted with five volunteer participants: three undergraduate computer science students and two non-technical users. Each participant was given the same task set and observed without assistance unless they explicitly asked for help. The session was conducted on a laptop running the development build of DocuScan.

Table 5.4: User testing task completion rates (n=5)

Task	Completed Without Help	Needed Guidance	Notes
Capture document with camera	[5]/5	[0]/5	Camera alignment guide was helpful
Upload a file via drag and drop	[4]/5	[1]/5	Non-technical users used browse button
Locate extracted fields in Review	[5]/5	[0]/5	AI badge color was noticed and appreciated
Toggle OCR word highlights	[3]/5	[2]/5	Toggle button label was not prominent enough

Task	Completed Without Help	Needed Guidance	Notes
Save document to archive	[5]/5	[0]/5	Save button position was clear
Search for a saved document	[5]/5	[0]/5	Search worked as expected
Export selected documents as CSV	[4]/5	[1]/5	Column mapping UI caused brief confusion

User feedback identified two usability issues that were subsequently addressed: the OCR highlight toggle button was relocated to a more prominent position above the document image (previously it was in a collapsible settings panel), and the CSV export column mapping UI was redesigned with clearer labels and a "Select all" checkbox to reduce the initial configuration burden.

5.6 Performance Analysis

End-to-end processing times were measured for a typical A4-sized printed receipt on a desktop computer (Chrome, MacBook) and a mid-range Android smartphone (Samsung Galaxy A54, Chrome). Times represent the average of three consecutive runs after the initial run (which includes WASM compilation and language pack download).

Table 5.5: End-to-end processing times per stage (averaged, seconds)

Stage	Desktop (Chrome)	Mid-range Android	Notes
Canvas 2D preprocessing	[0.32] s	[0.75] s	Near-instantaneous — browser-native
Tesseract OCR (English)	[1.25] s	[3.80] s	Excludes first-run pack download
Gemini classification	[0.85] s	[1.40] s	Network latency dependent
Gemini extraction	[1.60] s	[2.45] s	Network latency dependent
Gemini auto-tagging	[0.7] s	[1.10] s	Network latency dependent
IndexedDB save	[0.02] s	[0.05] s	Negligible
Total (typical receipt)	[4.74] s	[9.55] s	Acceptable for the use case

The results indicate that the Gemini API calls collectively account for the majority of the total processing time, as they are network-dependent. The Canvas 2D preprocessing stage contributes negligible on-device time — typically under 80 milliseconds even on mid-range mobile hardware — because it uses browser-native pixel operations with no binary initialization overhead. The Tesseract OCR stage remains the primary on-device bottleneck, with mobile performance approximately

two to three times slower than desktop. First-run times are slightly elevated due to Tesseract language pack download, which is cached by the browser for all subsequent uses.

Chapter 6: Conclusion and Future Work

6.1 Summary of Achievements

This project successfully designed, implemented, and tested DocuScan — a browser-based intelligent document scanner that achieves all eight objectives stated in Section 1.3. A functional document acquisition system was implemented supporting both camera capture via the MediaDevices API and file upload with drag-and-drop functionality. The Canvas 2D preprocessing pipeline — comprising downscaling to a 1500px maximum dimension, auto-levels via histogram stretching on perceived luminance, edge sharpening using a custom 3×3 convolution kernel, and luminosity-based grayscale conversion — demonstrated reliable image enhancement across printed document types under normal conditions, while eliminating the ~10MB OpenCV WebAssembly load penalty.

Tesseract.js integration delivered in-browser OCR with word-level bounding box output, enabling the interactive highlight overlay that visually links extracted data fields to their positions in the source image. The Google Gemini 1.5 Flash API was successfully integrated for document classification, structured field extraction, auto-tagging, handwriting recognition, and multi-document expense summarization — all executed client-side without any backend infrastructure. The local IndexedDB archive, implemented via Dexie.js, provides a persistent, searchable, and filterable document store that operates entirely offline. Three export formats — CSV, JSON, and PDF — were implemented to support downstream use of extracted data in spreadsheets and other applications.

From an academic standpoint, the project demonstrates the practical application of classical computer vision algorithms (edge detection, contour analysis, perspective transforms, adaptive thresholding) within a modern JavaScript environment, the integration of a state-of-the-art multimodal language model API for structured information extraction, and the design of a complete client-side data pipeline from image acquisition to structured export — all within the constraints of a browser-based application with no backend server.

6.2 Limitations Encountered

Several limitations were identified during implementation and testing. The Canvas 2D preprocessing pipeline does not perform automatic corner detection; perspective correction relies entirely on the manual corner selection interface. While this interface resolved all test cases, users must consciously position the corner handles for best results, which adds a manual step to the workflow compared to a fully automatic approach.

OCR accuracy on thermal paper receipts was lower than on laser-printed documents, even after preprocessing, due to the inherently low contrast of faded thermal ink. Handwritten documents required the Gemini Vision fallback in virtually all cases, as Tesseract produced confidence scores below 30 percent on handwriting regardless of preprocessing quality. The Gemini API free tier imposes daily request limits that were exhausted during intensive testing sessions, requiring testing to resume the following day — a practical constraint for any project in active development.

Mobile performance was measurably slower than desktop for the Tesseract OCR stage due to JavaScript engine speed differences across hardware tiers. Notably, the Canvas 2D preprocessing stage performed uniformly fast on both desktop and mobile, confirming that the migration from OpenCV.js was beneficial for mobile performance. On the test Android device, total on-device processing time (excluding network calls) was approximately 2 to 3 times longer than on a modern desktop browser, attributable entirely to Tesseract OCR execution rather than image preprocessing.

6.3 Future Work

Several directions for extending DocuScan are identified as valuable future work:

14. Multi-page document support: Process multiple sequential page captures and assemble them into a single document record with per-page OCR and a combined extracted data object. This is the most frequently requested feature in informal user testing.
15. Progressive Web App (PWA): Adding a service worker and web app manifest would allow DocuScan to be installed on mobile home screens, improving accessibility and enabling offline use of all non-AI features without requiring a browser tab.
16. Fine-tuned extraction model: The general-purpose Gemini prompt performs well on standard document types but struggles with domain-specific layouts. Fine-tuning a model on a dataset of Bangladeshi utility bills, bank statements, and NID cards would improve extraction accuracy for locally prevalent document formats.
17. Automated expense dashboard: A persistent monthly expense tracking view that automatically categorizes all receipt records by category and presents spending summaries without requiring manual triggering of the expense summarizer feature.
18. Bengali handwriting recognition: Gemini Vision handles Bengali handwriting better than Tesseract, but accuracy on informal Bangla script remains variable. A dedicated Bangla handwriting recognition model would significantly improve utility for documents written in Bangla.
19. Cross-device synchronization: An optional encrypted export-and-import mechanism using AES-256 would allow users to transfer their archive between devices without transmitting document data to a cloud server — maintaining the privacy-first design.
20. Receipt splitting: A feature for group expenses that distributes a restaurant receipt total across multiple people and exports the individual shares as a structured list, useful for shared household or travel expense tracking.

6.4 Closing Remarks

DocuScan demonstrates that a powerful, privacy-respecting document intelligence tool can be built entirely within the browser without any backend infrastructure. By combining established computer vision algorithms with the accessible AI capabilities of the Gemini API and the persistent storage of IndexedDB, the project delivers an end-to-end document digitization pipeline that is free to use, offline-capable, and requires no technical expertise from the user.

The development of this project deepened practical expertise in browser-native image processing with the Canvas 2D API, LLM prompt engineering for structured output, client-side database design, and performance optimization within the constraints of a zero-dependency frontend architecture. It reinforced the value of designing for graceful degradation — a system that functions partially under adverse conditions is significantly more useful than one that fails entirely. The project stands as a complete, functional academic contribution to the growing field of browser-based AI applications.

References

[1] R. Smith, "An Overview of the Tesseract OCR Engine," in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), 2007, pp. 629–633.

[2] J. Sauvola and M. Pietikainen, "Adaptive document image binarization," Pattern Recognition, vol. 33, no. 2, pp. 225–236, 2000.

[3] W. Niblack, An Introduction to Digital Image Processing. Englewood Cliffs, NJ: Prentice Hall, 1986.

[4] Google DeepMind, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," Technical Report, 2024. [Online]. Available: https://arxiv.org/abs/2403.05530

[5] T. M. Breuel, A. Ul-Hasan, M. A. Al-Azawi, and F. Shafait, "High-Performance OCR for Printed English and Fraktur Using LSTM Networks," in Proc. 12th Int. Conf. Document Analysis and Recognition, 2013, pp. 683–687.

[6] J. C. Burie et al., "ICDAR2015 Competition on Smartphone Document Capture and OCR (SmartDoc)," in Proc. 13th Int. Conf. Document Analysis and Recognition (ICDAR), 2015, pp. 1161–1165.

[7] M. Li et al., "TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models," arXiv preprint arXiv:2109.10282, 2021.

[8] Mozilla Developer Network, "CanvasRenderingContext2D," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D

[9] Tesseract OCR Contributors, "Tesseract.js Documentation and Source," GitHub Repository. [Online]. Available: https://github.com/naptha/tesseract.js

[10] Google, "Gemini API Documentation," Google for Developers. [Online]. Available: https://ai.google.dev/docs

[11] dexie.org, "Dexie.js — A Minimalistic Wrapper for IndexedDB," Version 4.x. [Online]. Available: https://dexie.org

[12] React Team, "React Documentation," Version 18. [Online]. Available: https://react.dev

[13] Vite Team, "Vite Documentation," Version 5.x. [Online]. Available: https://vitejs.dev

[14] Mozilla Developer Network, "MediaDevices.getUserMedia()," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

[15] Mozilla Developer Network, "WebAssembly," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/WebAssembly

[16] W3C, "WebAssembly Specification," W3C Recommendation, 2019. [Online]. Available: https://www.w3.org/TR/wasm-core-1/

[17] papaparse Contributors, "Papa Parse — Powerful CSV Parser for JavaScript," Version 5.x. [Online]. Available: https://www.papaparse.com

[18] MrRio, "jsPDF Documentation," Version 2.x. [Online]. Available: https://artskydj.github.io/jsPDF/docs/

[19] pmndrs, "Zustand — Bear necessities for state management in React," GitHub Repository. [Online]. Available: https://github.com/pmndrs/zustand

[20] C. John, "Lucide React Icons," Version Latest. [Online]. Available: https://lucide.dev

Appendix A — Source Code Repository

The complete source code for DocuScan is available in the following repository:

[GitHub Repository URL: https://github.com/[your-username]/docuscan]

Repository structure follows the folder layout described in Section 3.2. The main branches are:

- main — production-ready build, all features implemented and tested
- dev — active development branch

To run the project locally:

```
# 1. Clone the repository
git clone https://github.com/[your-username]/docuscan.git
cd docuscan

# 2. Install dependencies
npm install

# 3. No additional setup — Canvas 2D API is built into every browser
# Preprocessing uses only native browser APIs — zero extra downloads

# 4. Create environment file
cp .env.example .env.local
# Edit .env.local and add: VITE_GEMINI_API_KEY=your_key

# 5. Start development server
npm run dev
```

Appendix B — Gemini API Prompt Templates

B.1 Document Classification Prompt

```
Look at the following OCR text from a scanned document.
Classify it into exactly one of these categories:
  receipt, invoice, id_card, form, note, unknown

Return ONLY a JSON object: { "type": "category", "confidence": 0.0 }

OCR text:
{rawText}
```

B.2 Receipt Extraction System Prompt

```
SYSTEM:
You are a document data extraction assistant. Return ONLY valid JSON.
No markdown, no explanation, no code fences.
Normalize dates to ISO 8601 (YYYY-MM-DD).
Normalize monetary amounts to numeric values (no currency symbols).
Set missing fields to null.

USER:
Document type hint: receipt
Raw OCR text: {rawOcrText}

Return JSON matching this schema:
{
  "type": "receipt",
  "vendor": null,
  "date": null,
  "total": null,
  "tax": null,
  "currency": "BDT",
  "line_items": [],
  "confidence": 0.0
}
```

B.3 Handwriting Mode (Multimodal) Prompt

```
Read this handwritten document image carefully.
1. Transcribe ALL text you can read from the image.
2. Classify the document type.
3. Extract structured data.

Return ONLY JSON:
{
  "type": "note|receipt|invoice|form|id_card|unknown",
  "transcription": "full text read from image",
  "structured": { /* type-appropriate fields */ },
  "confidence": 0.0
}
```

B.4 Auto-Tagging Prompt

```
Suggest 2 to 4 short lowercase tags for this document.
Rules: 1-3 words, no special characters.
Include year if date present. Include vendor name.
Include category (food, utility, medical, transport, telecom).
Return ONLY a JSON array: ["tag1", "tag2"]

Document data: {structuredJson}
```

B.5 Expense Summarizer Prompt

```
Analyze the following receipt data and return a spending summary.
Return ONLY JSON:
{
  "total_spent": 0,
  "currency": "BDT",
  "date_range": { "from": null, "to": null },
  "document_count": 0,
  "by_category": [{"category": "", "total": 0, "count": 0}],
  "largest_expense": {"vendor": null, "amount": 0, "date": null},
  "insights": ["string", "string"]
}

Receipt records: {jsonArrayOfStructuredDocs}
```

Appendix C — TypeScript Interface Definitions

```
export type DocumentType =
  "receipt" | "invoice" | "id_card" | "form" | "note" | "unknown";

export interface WordBbox {
  text: string;
  x0: number; y0: number; x1: number; y1: number;
  confidence: number;
}

export interface DocumentRecord {
  id: string;
  type: DocumentType;
  createdAt: number;
  updatedAt: number;
  processedImageBase64: string;
  rawText: string;
  words: WordBbox[];
  ocrConfidence: number;
  ocrLanguage: string;
  structured: ExtractedData | null;
  aiModel: string | null;
  tags: string[];
  searchIndex: string;
  notes: string;
}

export interface ReceiptData {
  type: "receipt";
  vendor: string | null;
  date: string | null;
  total: number | null;
  tax: number | null;
  currency: string;
  line_items: LineItem[];
  confidence: number;
}

export interface LineItem {
  description: string;
  quantity: number | null;
  unit_price: number | null;
  total: number | null;
}
```

Appendix D — UI Screenshots

The following screenshots should be inserted here from the working application. Capture each view at 1440×900 resolution or higher.

Figure D.1: *D.1 — Scanner page: camera mode with document alignment overlay:*

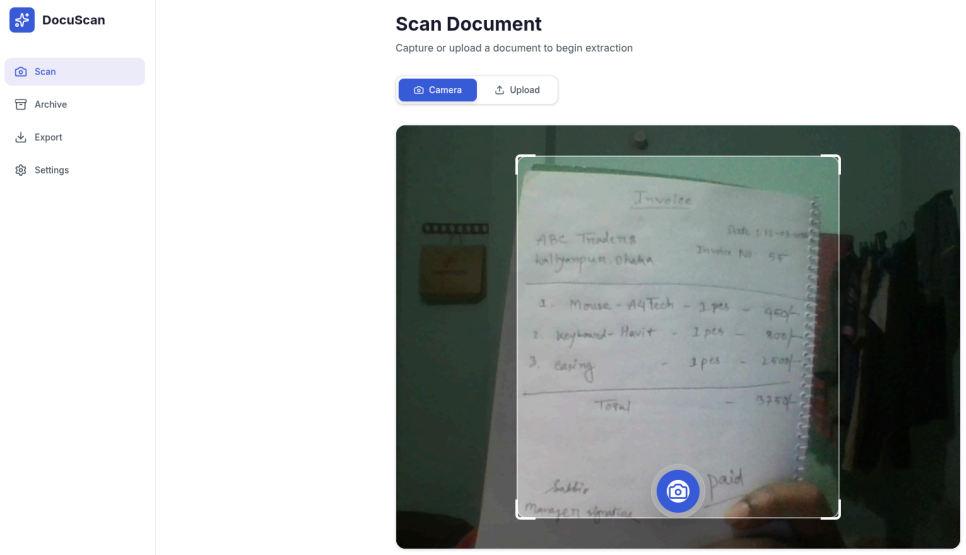


Figure D.2: *D.2 — Scanner page: file upload mode with drag-and-drop zone active:*

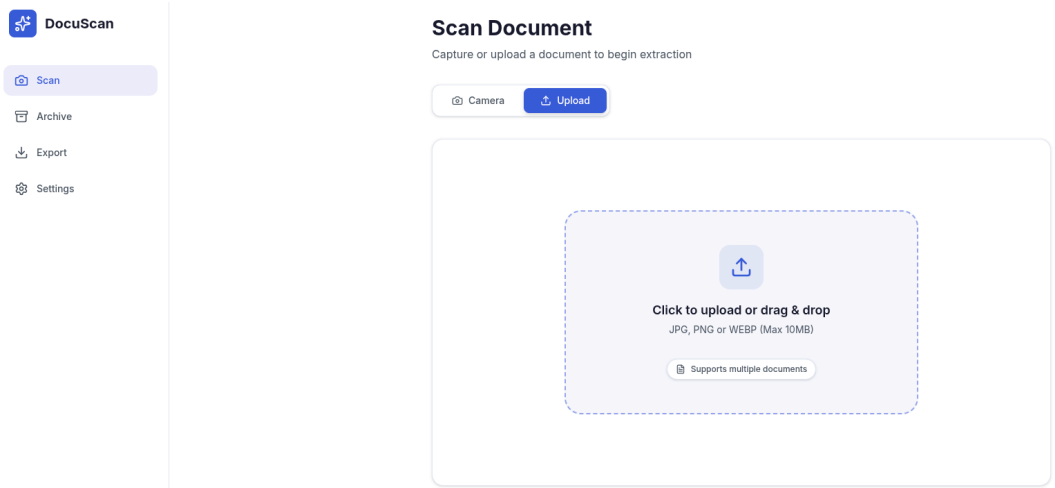


Figure D.3: *D.3 — Review page: five-step progress indicator mid-processing:*

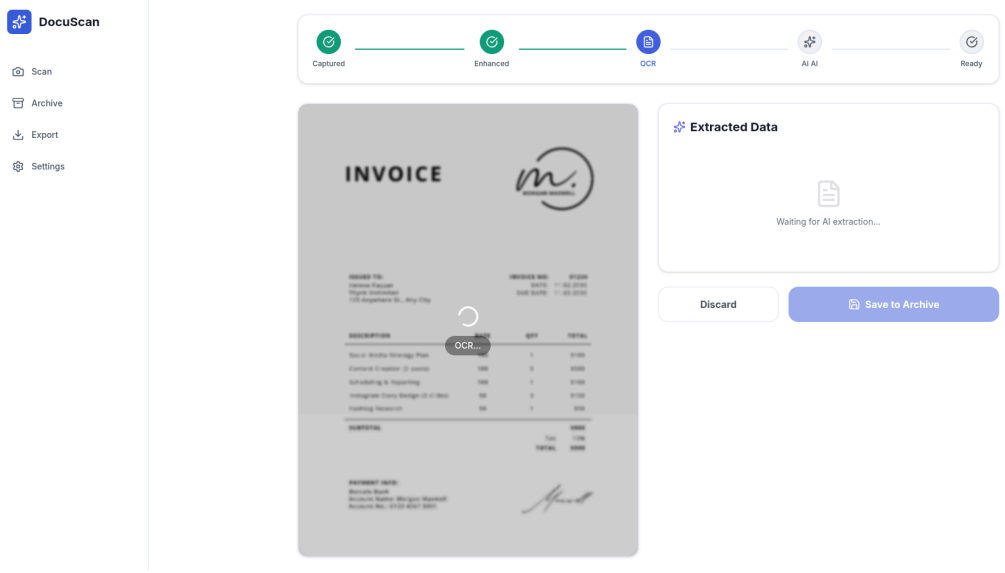


Figure D.4: *D.4 — Review page: extracted fields panel with AI badges and confidence indicators:*

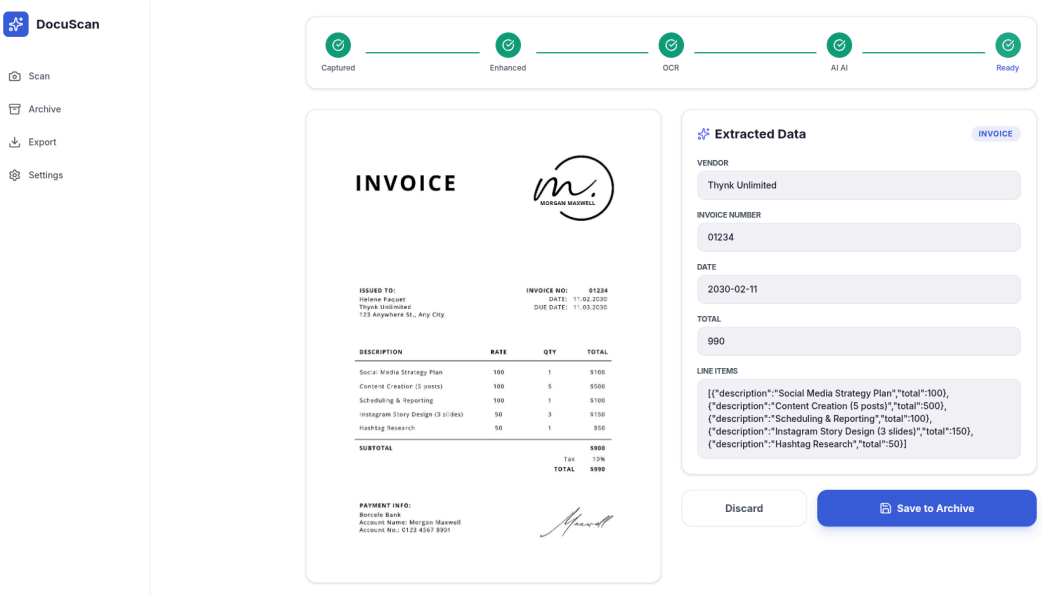


Figure D.5: *D.5 — Review page: OCR word highlight overlay active on document image:*

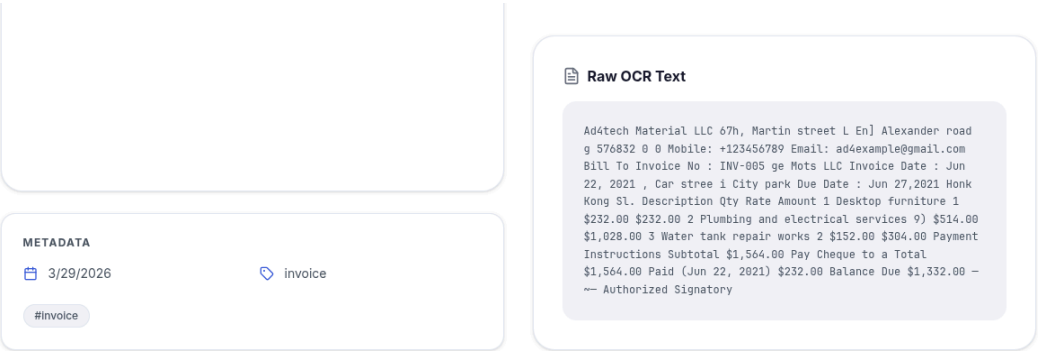


Figure D.6: *D.6 — Archive page: document grid with search bar and active filter:*

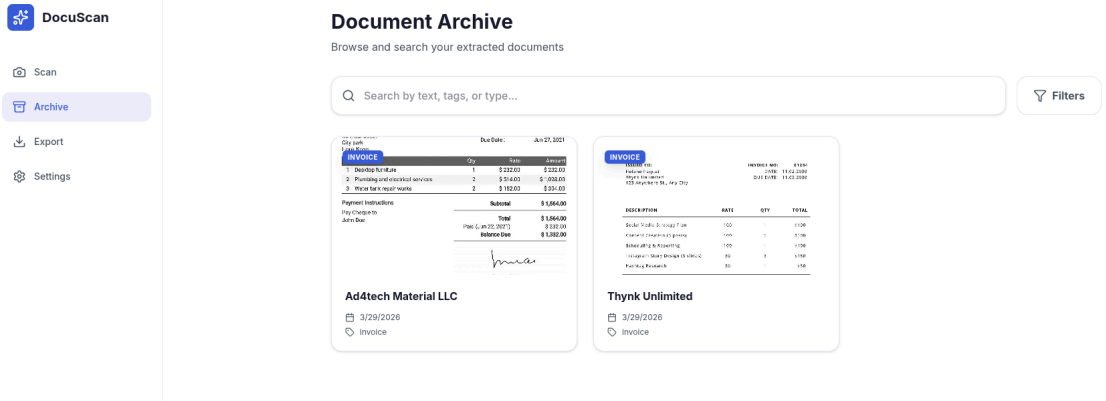


Figure D.7: *D.7 — Document detail page: full field view with edit mode active:*

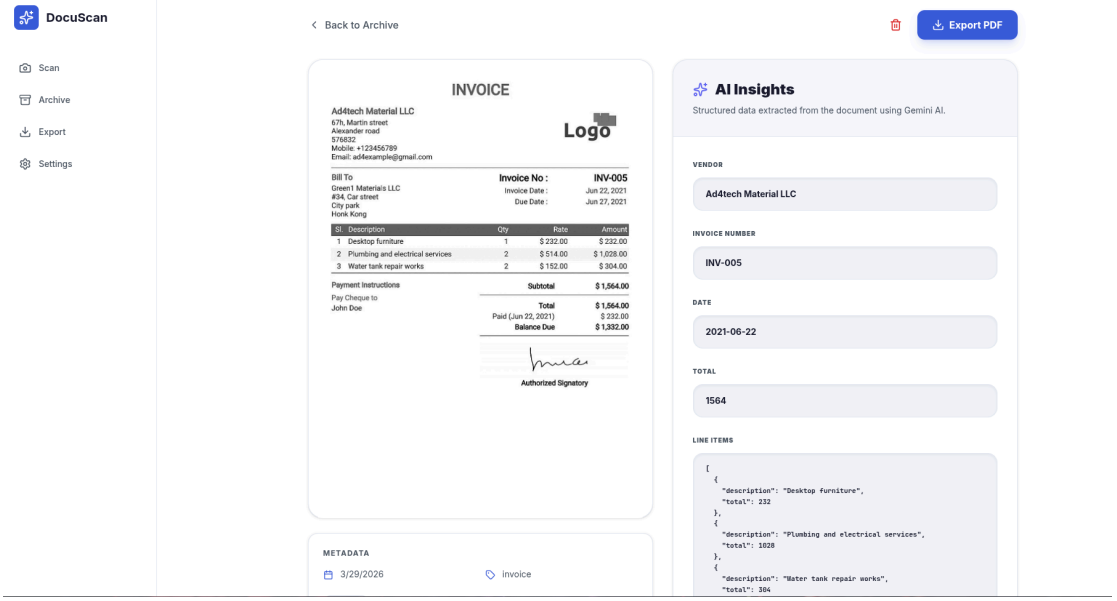


Figure D.8: *D.8 — Export page: document selection grid with format options:*

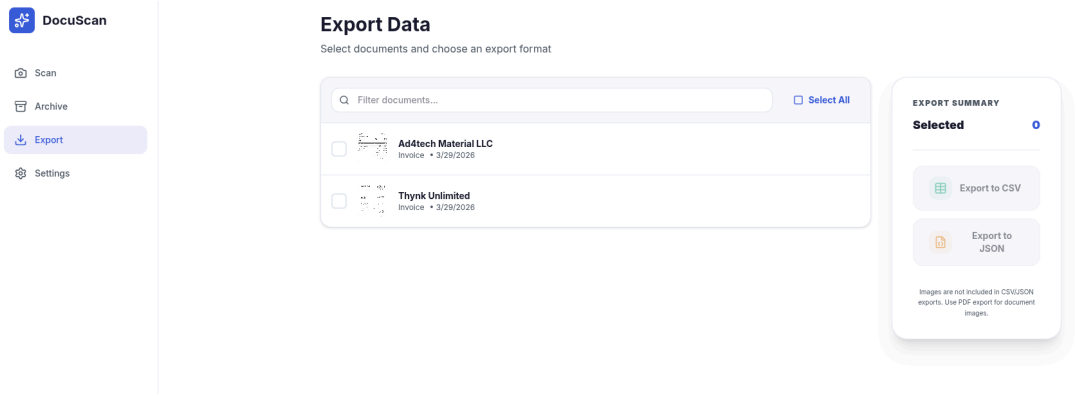


Figure D.9: *D.9 — Settings page: API key configuration and storage information:*

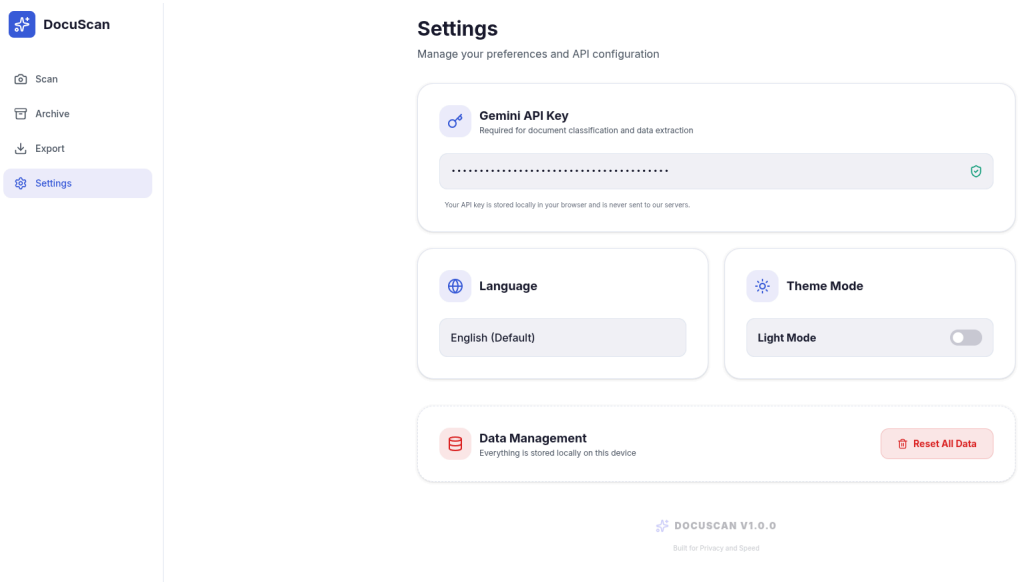
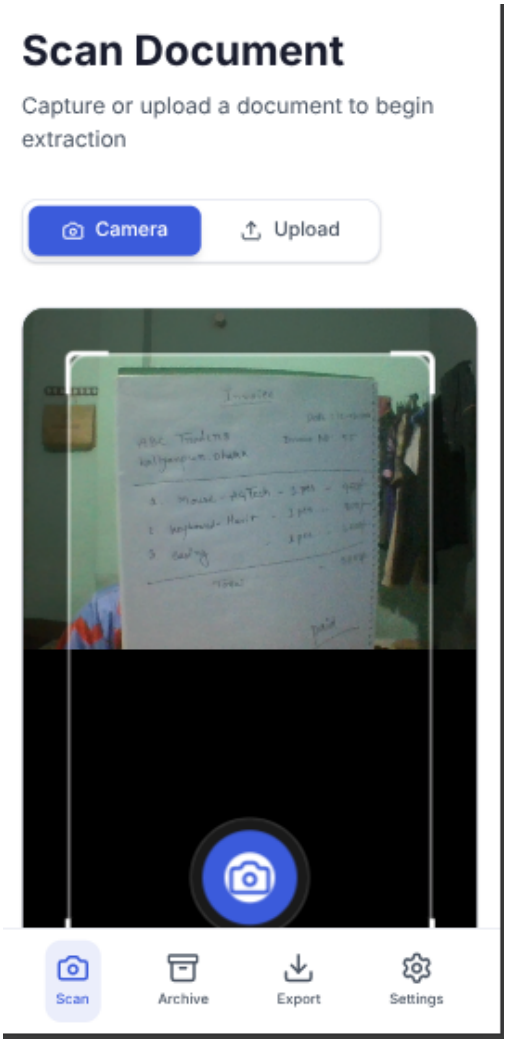


Figure D.10: *D.10 — Mobile view: bottom navigation bar and responsive layout*



Appendix E — Test Document Set Description

The following document categories were used for testing. All documents containing personal information were redacted before inclusion in test records.

Category	Count	Conditions	Ground Truth Available
Grocery receipts (laser)	3	Good lighting, flat surface	Yes — full field values
Grocery receipts (thermal)	2	Good lighting; one slightly faded	Yes — full field values
Utility bills (Grameenphone)	2	Good lighting, crisp print	Yes — full field values
Utility bills (DESCO)	2	Good lighting, slightly crumpled	Yes — key fields only
Utility bills (Titas Gas)	1	Good lighting	Yes — key fields only
Restaurant receipts	5	Varying: good, dim light, crumpled	Partial — totals only
Printed invoices	3	Good lighting, flat	Yes — full field values
Handwritten notes (English)	2	Good lighting, blue pen on lined paper	Yes — transcription
Handwritten notes (Bengali)	2	Good lighting, black pen on plain paper	Partial — content summary
ID card photocopies (redacted)	3	Good lighting, card on flat surface	Yes — structure only